Assignment 1 - Phase 2: Pond Catchment Analysis Backend

Name: Roshan Raj
Roll Number: 12341830

1. GitHub Repository

The complete source code for this assignment is maintained in this GitHub repository: https://github.com/roshanraj9136/village_pond_planner

2. Working API Route & Documentation

The backend is built using FastAPI. To use the API, run the server locally on port 8000 (`uvicorn main:app --reload`).

- **Working API Route URL:** `http://localhost:8000/analyzeContour` (Method: `POST`)
  - *Alias Route:* `http://localhost:8000/findCatchment` (Method: `POST`)
- **Input Expected:** A `.kml` or `.kmz` file containing contour map data, uploaded as `multipart/form-data` with the key `file` .
- **API Documentation:** The interactive Swagger UI documentation generated by FastAPI is available at `http://localhost:8000/docs` . This documents the request parameters, schemas, and response formats.

3. Catchment Estimation Approach

To ensure the backend works dynamically for any terrain, I implemented a robust 5-step approach to estimate the catchment area:

1. **Extracting KML Data:** The code parses the uploaded XML/KML structure to extract geographic coordinates (latitude and longitude) and their corresponding elevation values. It can pull elevation from `<name>` , `<description>` , or the Z-index of the coordinates.
2. **Digital Elevation Model (DEM) Interpolation:** Since a contour map only has data along the lines, there are gaps between them. I used SciPy's cubic interpolation

(`scipy.interpolate.griddata`) to fill in these gaps, creating a continuous 2D grid of the terrain (a DEM). I also applied a Gaussian filter to smooth out sharp, unrealistic spikes.
3. **Terrain Slope & Flow Direction (D8 Algorithm):** By calculating the gradient across the grid, the code determines the slope. I then implemented the standard GIS **D8 Flow Direction Algorithm**, which analyzes every cell's 8 neighbors and calculates the steepest direction of water flow.
4. **Flow Accumulation & Pond Siting:** A **Flow Accumulation** matrix is generated to count exactly how many upstream cells flow into any given cell. The most suitable pond location (the "Pour Point") is dynamically identified as the terrain cell with the absolute maximum flow accumulation, guaranteeing it is the ultimate natural sink of the basin.
5. **Catchment Delineation & Runoff:** The code backtracks from the Pour Point up the Flow Direction matrix to map out every single contributing cell, establishing the exact watershed catchment boundary. The total area is then used in the Rational Method to calculate volumetric runoff and pond capacity.

4. Demonstration using the Sample Map

I tested the API using the provided `contours_1m.kml` sample map. I made a `POST` request to `/analyzeContour` with the file attached.

Here are the results the API dynamically calculated from the map: * **Contours Processed:** 92 contour lines parsed. * **Elevation Range:** 267.33 meters to 292.89 meters. * **Average Terrain Slope:** 7.50% (Classified as Rolling). * **Optimal Pond Location:** Latitude 21.262967, Longitude 81.286227. * **Estimated Catchment Area:** 5.49 Hectares (54,901.84 square meters). * **Estimated Runoff Volume:** 43,795.19 cubic meters.

The API successfully returned these results in a structured JSON format, along with a GeoJSON Polygon representing the exact boundary of the catchment area so it can be drawn on a frontend map.

5. Code Extensibility (Future Phases)

A major requirement for this phase was to ensure the code is extensible and not hard-coded to the sample map. Here is how I achieved this for the 10 points: * **No Hard-coded Bounds:** The system automatically calculates the bounding box (`min_lat` , `max_lat` , `min_lng` , `max_lng`) based strictly on the data points inside the uploaded KML file. * **Adaptive Grid Scaling:** The distance calculations automatically adjust for latitude distortions using equirectangular projection math, so the area calculation works accurately for a map located anywhere in the world. * **Dynamic Parsing:** The KML parser doesn't rely on strict formatting. It uses regex and XML traversal to find

elevations regardless of how the GIS software exported the file. * To prove this extensibility, I tested the API with a completely unseen, massive 6.7 MB contour map. The backend processed it perfectly without a single line of code changed, delineating an 874-hectare catchment area. This confirms the logic is fully generalized for any future contour maps.